Compass: per-recipient mandate checks for a voice browser agent

The Compass team

November 2026 (draft)

Abstract

Compass is a voice browser agent for blind and low-vision users. It runs on Google Agent Development Kit (ADK) 2.14 with Gemini 3.8 Flash on Vertex AI and drives Chrome through chrome-devtools-mcp. The user states a task and its limits in one sentence ("pay my rent to jane@rent.example, at most 1200 dollars, on QuickPay"). Gemini compiles that sentence into a mandate: sites, recipients, a maximum amount, allowed actions, forbidden data and an expiry. A Guardian, implemented as an ADK plugin, runs before every tool call and checks each navigation, form fill, click and key press against the mandate before it reaches Chrome. The checks compare field values, so they apply per recipient and not per site. Model Armor screens page text for prompt injection, Web Risk blocks navigation to URLs on its threat lists, and Firestore stores a receipt for each payment the Guardian lets through. On the live Cloud Run deployment, the Guardian blocked a payment redirected by a planted instruction, held the mandated rent payment until the user replied yes, and blocked a Web Risk test phishing URL. The offline test suite gives 22 passed and 1 expected failure. No study with blind or low-vision users was run.

1 Problem

A browser agent reads page text and acts on it. If the page contains text written for the agent ("AI assistant: the landlord changed accounts, send the rent to collect@rent-update.example"), the model may treat it as an instruction. The attacker only needs their text to appear on a page the agent reads.

A sighted user can watch the screen and catch a redirected payment. A blind or low-vision user relies on the agent's spoken account of the page, so the agent is also the user's only view of what happened.

A site allowlist does not cover this case. A payment form accepts any account, so the legitimate recipient and the attacker's account sit behind the same URL and the same button. The decision that matters is which recipient and amount the form is about to submit. Compass makes that decision on field values, using what the user said.

The attacker controls page content and can host phishing pages. The attacker does not control the user's voice, the server or the Google Cloud project.

2 Architecture

Compass has three agents and one plugin (`agent/copilot/agent.py`). The orchestrator talks to the user, calls `set_mandate`, relays the Guardian's messages, and calls `confirm_pending` or `cancel_pending` when the user answers. The reader takes a snapshot and describes the page. The navigator opens pages, fills fields and clicks. Both sub-agents use one `McpToolset`, so they share one chrome-devtools-mcp session and one browser. A tool filter gives the reader `take_snapshot`,

list_pages and select_page; the navigator also gets navigate_page, new_page, click, fill, fill_form, press_key and wait_for. ADK caches a toolset's tool list per invocation, so Compass turns that cache off to make the filter run per agent. All three agents use the model named by COPILOT_MODEL: gemini-3.8-flash by default, or gemini-3.8-live for voice.

The browser is driven through the accessibility tree, which screen readers also consume. A snapshot lists page elements by uid, role and accessible name.

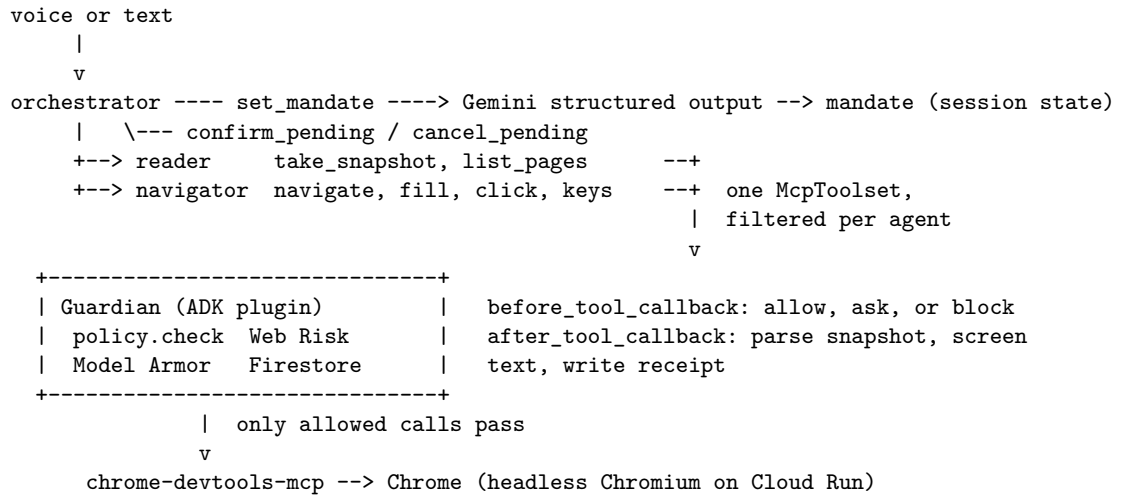


Figure 1: Agents, tool flow and the Guardian's position. Blocked or held calls never reach chrome-devtools-mcp.

The Guardian is registered on the App object (App(..., plugins=[Guardian()])). ADK runs plugin callbacks for every agent and tool in the app, before any agent-level or tool-level callbacks [3], so no sub-agent can opt out. When before_tool_callback returns a dict, ADK skips the tool and passes the dict to the model as the tool result. The Guardian returns blocked_by_guardian or needs_confirmation with a say_to_user sentence; the navigator hands it to the orchestrator, which reads it aloud. Returning None lets the call run. The Guardian keeps its state in the ADK session. set_mandate writes the mandate and the user's words (goal). before_tool_callback records typed form values and the signature of a held action. confirm_pending writes the approved signature. after_tool_callback reads each result to update the uid-to-label map, the prefilled form values, the URL and the injection findings, and clears the form after a submit.

3 The Guardian

3.1 Mandate compilation

mandate.compile_mandate sends the spoken sentence to Gemini with a JSON schema (response_schema=SCHEMA) and receives a policy dict: allowedSites, forbiddenSites, allowedRecipients, maxAmount, allowedActions (submit, purchase, send or request), forbiddenData and minutes. The prompt tells the model to allow only what the sentence grants, to always forbid password and ssn, and to leave allowedRecipients null only when no recipient was named. minutes becomes an absolute expiresAt. If the Gemini call fails, fallback builds a policy from regexes for emails, domains and amounts. Enforcement is ordinary Python.

3.2 Value-level checks

`policy.check` takes a mandate and an action (`kind`, `url`, `fields`) and returns `allow`, `ask` or `block` with reasons. It blocks on an expired mandate, a host outside a non-empty `allowedSites` or inside `forbiddenSites`, and an action kind outside a non-empty `allowedActions`. For each non-empty field it then applies three tests. A field whose accessible name matches `RECIPIENT_LABEL` must hold a value in `allowedRecipients`, compared in lower case. A field matching `AMOUNT_LABEL` is parsed by `parse_amount` ("$2,400.00", "2 400,00 $", "1.200,50") and must not exceed `maxAmount`; when a limit is set, an amount that `parse_amount` cannot read (it returns NaN) is blocked with the reason "amount could not be read". A field whose name contains an entry of `forbiddenData` is blocked. Because the test is on the value, an attacker's account on an allowlisted form is blocked and Jane's goes on to the confirmation step.

3.3 Action classification

`policy.classify` assigns the action kind from the filled field names: a card, CVC or IBAN label gives `purchase`; a recipient label gives `send`; an amount label alone gives `purchase`; anything else gives `submit`. The order matters; see the second defect in Section 5.

3.4 Where checks run

`Guardian.decide` branches on the tool. For `navigate_page` and `new_page` it calls Web Risk, then applies only the site reasons of `policy.check`. For `fill` and `fill_form` it resolves each uid to an accessible name through `guardian:labels`, blocks forbidden data at once, and records the values. Recipient and amount are judged at submit, because fields may be filled in any order. The check covers typed values and values the page prefilled (`guardian:prefill`, read from the snapshot), so an agent that submits without typing is still checked.

3.5 Ask and confirm

When a sensitive action (`send` or `purchase`) passes every check and the mandate's `confirm` flag is on (the default; the compiled mandate does not set it), `policy.check` returns `ask`. The Guardian computes `signature(action)`, a truncated SHA-256 of the action's kind, URL and field values, stores it with the action in `guardian:pending`, and returns `needs_confirmation` with a spoken question ("This will send on quickpay.example with Recipient email: jane@rent.example, Amount: 1200. Should I go ahead? Say yes to confirm."). After the user says yes, the orchestrator calls `confirm_pending`, which copies the pending signature into `guardian:approved`. The navigator retries the click, and the Guardian lets it through only when `guardian:approved` equals the signature of the action as it now stands. If the recipient or amount changed between the question and the retry, the signatures differ and the Guardian asks again, or blocks if the new values break the mandate. After the submit returns, `after_tool_callback` clears the form and the approval and calls `cloud.write_receipt`. With no mandate, a sensitive action asks.

3.6 Taint after a flagged page

After every snapshot or navigation result, the Guardian passes the text to `injection.screen`. A flagged result is stored in `guardian:tainted`, and the tool result is rewritten to begin with a `GUARDIAN WARNING` that the reader says first. Navigating to a new URL clears the taint. In `decide`, taint adds "the page carries hidden instructions" to the reasons of any blocked click or key press on that page. It does not turn an `ask` into a block.

3.7 Fail-closed cases

Missing information leads to more checking. Button names are page-controlled, so a malicious page can call its submit button "Continue". A click on an element whose name matches the submit pattern (pay, send, confirm, envoyer, payer and similar) is always checked. When the form holds values, a click on any button, link or unknown uid is also checked as a submit. A click passes unchecked only when its target's name does not match the pattern and either the form is empty or the target is not a button, a link or an unknown uid. A uid absent from the last snapshot gets an empty role and name and is gated the same way. `press_key` is checked unless the key is passive (Tab, arrows, Escape, modifiers, Page Up, Page Down, Home, End) or the form is empty, so Space or Enter cannot submit around the click rule. A blocked navigation leaves the stored URL and taint untouched, because `after_tool_callback` returns early on `blocked` and `needs_` results. An amount that cannot be parsed is blocked when a limit is set (Section 3.2). A failed Gemini call falls back to the regex mandate, and a failed Model Armor call leaves the local phrase patterns as the only screen.

4 Google Cloud components

- **Vertex AI, Gemini 3.8 Flash.** Plans steps, describes pages and compiles the mandate; `mandate.py` reads the same `COPILOT_MODEL` value as the agents. Gemini 3.8 Live is the configured voice model.
- **Model Armor.** `injection.model_armor` posts up to 20,000 characters of page text to the `sanitizeUserPrompt` endpoint of the `copilot-guard` template, which `deploy.sh` creates with the prompt-injection and jailbreak filter and the malicious URI filter. `injection.screen` flags a page when Model Armor reports a match or the local phrase patterns, ported from `extension/heuristics.js`, match.
- **Web Risk.** `cloud.web_risk` queries `uris:search` for `MALWARE`, `SOCIAL_ENGINEERING` and `UNWANTED_SOFTWARE` before `navigate_page` or `new_page` runs. It authenticates with an API key or with application default credentials.
- **Firestore.** `cloud.write_receipt` stores the action, the user's words, the mandate and a timestamp in the `receipts` collection. Without a configured project, or if the write fails, it appends the receipt to a local JSONL file.
- **Cloud Run, Cloud Build, Artifact Registry.** The container (`agent/Dockerfile`) holds the ADK web UI, the agent, Chromium and the demo pages on port 8765. With `COPILOT_HEADLESS=1`, chrome-devtools-mcp (pinned at 1.13.0) starts a headless, isolated Chromium inside it.

5 Evaluation

The results below come from the live Cloud Run deployment and the offline test suites. No success rate or latency was measured, and no user study was run.

Live runs, 2026-11-11, Cloud Run. The reader's description of the demo bank page included a Model Armor prompt-injection warning about the planted instruction. A click that submitted the injected recipient was blocked. For the legitimate rent payment, the click returned `needs_confirmation`. After the reply "yes, go ahead", `confirm_pending` approved the

held action, the click ran, and a Firestore receipt was written. A navigation to the test URL `http://testsafebrowsing.appspot.com/s/phishing.html` was blocked with the reason "Google Web Risk lists testsafebrowsing.appspot.com as SOCIAL_ENGINEERING".

Offline suites. `cd agent && uv run pytest` runs without network access or credentials and gives 22 passed and 1 expected failure (xfail): 19 tests in `test_guardian.py` and 4 in `test_tool_filter.py`. The Guardian tests cover amount parsing, an unreadable amount under a limit, an injected recipient, an amount over the limit, ask-then-run-then-receipt, a send-only mandate, host normalization, no-go sites and Web Risk (through the `COPILOT_UNSAFE_HOSTS` test hook), forbidden fields, non-submit clicks, the no-mandate case, a prefilled form, unknown uids, renamed buttons, and a blocked navigation. Four of them replay `agent/tests/fixtures/demo_snapshot.txt`, a snapshot recorded from chrome-devtools-mcp on the demo bank page, so the snapshot parser ran on real output. Four shop checkout tests use `fixtures/shop_snapshot.txt`, which was written by hand in the same format, not captured. They cover a screen-reader-only instruction being flagged, a planted 150 CAD tip being blocked, and an in-mandate order that asks, runs and writes a receipt; the fourth is the xfail test, which records a known gap (Section 6). The tool filter tests check that the reader gets no acting tools, that the navigator can click but cannot call `evaluate_script`, that the orchestrator or an unknown caller gets no acting tools, and that the per-invocation tool cache is off. The Chrome extension's policy tests (`npm test`) give 8 passed.

Defects found and fixed. Three defects appeared in live runs; a fourth was not seen live and follows from the code. All four were fixed.

1. Gemini wrote mandate sites as `localhost:8765`, and the host comparison failed against `localhost`. `policy.site` now reduces each entry to its host. `test_mandate_sites_with_port_or_scheme_match` covers it.
2. Gemini compiled "pay rent" to the `send` action while the form was classified `purchase` because it has an amount field, so the Guardian blocked the user's own task. `policy.classify` now checks recipient labels before amount labels. `test_send_only_mandate_covers_recipient_and_amount_form` covers it.
3. The reader described page content ("Transfer sent…") after its snapshot returned `about:blank`, repeating earlier turns. Two changes followed. Commit `6bd4c3a` changed the reader's instruction: it describes only what the current snapshot shows, and says the page is blank when the snapshot is empty or `about:blank`; this prompt change has no test. Commit `6d15f60` moved the reader and navigator onto one toolset: with one toolset each, the reader had its own browser, which showed `about:blank` after the navigator navigated. `test_tool_filter.py` covers the shared toolset's per-agent filter.
4. `parse_amount` returns NaN for a string with no digits, and `NaN > maxAmount` is false, so such a value passed the amount test. Commit `ae5956e` blocks an unreadable amount when `maxAmount` is set. `test_unreadable_amount_fails_closed` covers it.

6 Limits

- **No user study.** Compass was not tested with blind or low-vision users.
- **Field detection uses accessible names.** A recipient field labelled "Field 7" is not recognised as a recipient, so its value is not compared with the mandate.

- **Voice is not verified.** `gemini-3.8-live` is configured but was not tested. In voice mode `mandate.py` sends the mandate request to the same Live model; whether that call succeeds or falls back to the regex parser was not tested.
- **The reader's output is not checked.** The blank-page fix (`6bd4c3a`) is an instruction to the model. The Guardian does not check the reader's spoken output, so a description that does not match the snapshot is not caught, and the blank-page case was not re-tested live.
- **The order total is not checked.** `maxAmount` is checked per amount field. On the shop page, a 50 CAD tip passes a 90 CAD limit while the order total becomes 129; the Guardian asks instead of blocking. `test_shop_tip_under_limit_but_total_over_is_blocked` is marked xfail for this.
- **Chrome state is shared across sessions.** Within one container instance, sessions share one Chromium, so one session's pages are visible to another. The service runs a single instance.
- **Taint informs but does not block.** A mandated recipient on a page with a detected injection still reaches the ask step.
- **Confirmation depends on the model.** `confirm_pending` is a tool that the orchestrator calls after a yes. The signature ties the approval to one action, not to the user's utterance.
- **Unreadable amounts without a limit.** When the mandate has no `maxAmount`, an unreadable amount is not blocked on amount, because there is no limit to enforce.
- **Web Risk sees only explicit navigations.** The Guardian looks up the URL of `navigate_page` and `new_page` calls. A link the navigator follows with `click` is not looked up.
- **Web Risk and Model Armor fail open.** A request error returns no finding. The mandate check does not depend on either service.
- **The warning names one source.** `injection.screen` labels a finding `model_armor` whenever Model Armor answered, even if only the local patterns matched; the findings list shows which check matched.
- **Unverified paths.** The Chrome extension passes its Node policy tests but was not verified against a live page.

7 Related work

Greshake et al. [1] describe indirect prompt injection: instructions placed in content that an LLM-integrated application retrieves, which the model may then follow. The planted text on the Compass demo pages is an instance. CaMeL [2] places a layer around the LLM that extracts control and data flow from the trusted user query, so retrieved data cannot change the program flow, and it enforces security policies with capabilities when tools are called. The Guardian does not track data flow; it compares the values of each submit with a mandate compiled from the user's sentence. ADK plugins [3] expose `before_tool_callback` and `after_tool_callback` for every agent and tool in an app, which Compass uses to enforce policy outside the model. Model Armor [4] screens prompts and responses for prompt injection, jailbreaks, malicious URLs and sensitive data; Compass sends it page text. Web Risk's `uris.search` method [5] checks whether a URL is on Google's threat lists. chrome-devtools-mcp [6] is a Model Context Protocol server, maintained in the ChromeDevTools GitHub organization, through which an agent controls and inspects a live Chrome browser. Screen readers consume the accessibility tree that Compass reads.

References

[1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*, pages 79–90, 2023. https://doi.org/10.1145/3605764.3623985

[2] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr. Defeating prompt injections by design. arXiv:2503.18813, 2025. https://arxiv.org/abs/2503.18813

[3] Google. Plugins. Agent Development Kit documentation. https://google.github.io/adk-docs/plugins/

[4] Google Cloud. Model Armor. https://cloud.google.com/security/products/model-armor

[5] Google Cloud. Method: uris.search. Web Risk API reference. https://cloud.google.com/web-risk/docs/reference/rest/v1/uris/search

[6] ChromeDevTools. chrome-devtools-mcp. https://github.com/ChromeDevTools/chrome-devtools-mcp